

Landing The Nostromo

Alan Sutcliffe

THE LANDSCAPE

In the summer of 1978 I was working with System Simulation Ltd. in London. They were asked to produce several sequences of computer animation for a space-fiction feature film already being made by 20th Century Fox. I had heard of one or two people connected with it: Ridley Scott was director and one of the actors was Ian Holm. A lot of Americans were also involved.

It turned out the film was called *ALIEN*, but that did not mean much to us then. I was given a brief look at an early script and asked to work on a section where the space-craft was coming in to land on an unknown planet. The atmosphere was opaque but some kind of future radar allowed a computerized view of the terrain on a screen in the command deck. I imagined, in my simple way, that there would be dialogue to put this over

Hey, look at the Ramdac screen, Skipper.

Looks damn hilly down there.

Here's a flattish patch.

That'll have to do.

Obviously I had seen too much Star-Trek; *ALIEN* was much more subtle than that. No explanations, no apologies. But I didn't know that—and I didn't need to know it—until I saw the film a year later. I was asked to make the computer output look like computer output, to make 30 seconds of landing in rugged country with a flat area.

The screen was to be surrounded by meaningful-looking animated displays showing altitude and angles and such.

Alan Sutcliffe, 4 Binfield Rd., Wokingham Berkshire, RG11 1SL England.

The hardware to be used was a Prime 300 connected to an FR 80 from 111 which can plot directly onto cine-film. Running on this system at the SRC laboratories in Oxfordshire was the software animation package, Frolic, developed by C. Emmett.

My program was written in Fortran with calls to Frolic subroutines. Apart from general interest, you might ask why write about this in *Creative Computing*? A system costing over £1/4m is nobody's personal computer. The answer is that the two main parts of the program that I shall describe, the generation and display of the landscape and putting up the character displays, can both be realised on a small machine, provided that it drives a device with vector graphics.

Magic Mountains

"Magic Mountains" was my pet name for the project. The original scheme was to digitize a model landscape, but one day a block of about two cubic yards of polystyrene turned up in the office, its top carved into all sorts of fantastic shapes. Its overhangs and near-caves would have defeated the method I had planned to perform hidden line removal. In any case it was decided that it would be too time-consuming and too tedious to measure this model in detail and enter several thousand data values to represent it. I would synthesize the mountains from nothing. The whole process for the landscape consisted of three main stages:

1) *Generation.* On an equally spaced x-y grid of 50 x 50 points, a value of z was generated for the height of the ground at each point.

2) *View point.* This three-dimensional information was converted, for each of a series of angles and distances, into two-dimensional positions on the display screen. The series of angles corresponded to the view from the descending spaceship. The conversion and the control loop for the animation were performed by calls to Frolic, and will not be described here in detail.

3) *Hidden line removal.* For each view, the scenery was displayed by drawing a line through each set of points with the same x-values in the original three-dimensional form, but leaving out any

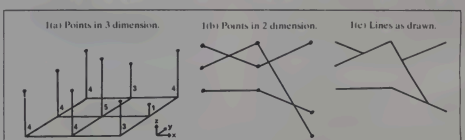


Figure 1. Three forms of the landscape data.

Nostromo, continued...

lines that were visually below the horizon formed by the nearer lines. On this scheme, a flat plain would be represented by a set of parallel lines running across the screen. Finally the lines had to be plotted, a trivial step using Frolic.

Figure 1 shows the form of the data after each of these three steps for a sample part of the grid.

Generating the Landscape

There were two parts to the landscape, an underlying terrain and an arrangement of mountains laid on top of it. I used the sum of four separate mathematical expressions to make the basic structure. There was a valley running from front to back achieved by applying a flattened sine-wave. There were three small ridges running diagonally across the area. From front to back the ground first rose then flattened out and rose again, indicated by

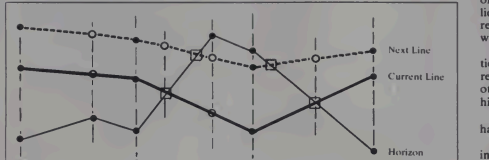


Figure 5. How intermediate points arise on the horizon.

a bunching together of the lines in the upper-middle of the pictuere. This area was overlaid with mountains that it hardly showed in the final version. Notice that applying an equal slope all the way from front to back would have no effect except to alter the apparent angle of view—all that matters is the relative slope in different parts of the picture. The fourth term in the expression was a small random perturbation of each point, to add a little texture and interest.

Figure 2 shows the landform that resulted from this calculation. It also shows an example of one of the hills on top of this. There were 400 hills altogether. Each normal hill covered an area of 5 x 5 grid points, and had a central peak with roughly symmetrical sides. About 20% of the hills were made into mountains covering 7 x 9 grid points, with a higher limit on the size of the peak. About 15% of the hills were made asymmeterical, and a further 20% were made into holes rather than hills by changing the sign of the expression.

The placing of the hills was controlled to give a roughly flat mountain-free area at the front of the picture, and also so that the hills at the back tended to be somewhat higher than those at the front. Apart from this, the locations and sizes of the hills and their detailed shapes were determined by calls to the random function. Figures 3 or 4 show two views of the final landscape. Since two or more hills could overlap there are some double peaks, flat tops, and small ridges and groups of peaks. These came by chance and it was not necessary to program them separately.

It may be thought that this method was over-elaborate. But I have found in the past that in simulating both sounds and appearances, a wealth of detail lends an air of verisimilitude. Geology, like much else in the world, is the result of a rich variety of complex processes. Some of the detail may be hardly perceptible, like the three ridges which can only be seen at the edges of the picture. Some, like the rise and flattening of the land already noted, may not be seen at all. This does not matter. The computation is only performed once, and no harm is done save a small amount of redundant coding and computing. In this case, neither mattered. The whole routine to generate the landscape took less than 40 lines of Fortran. Using a small system on which time and space would be more important, it would be easy to weed out the redundant parts.

The main change I would be likely to make implementing something like this on a small system would be a reduction in the size of the 50 x 50 array, depending on the storage available, the type of display hardware and the purpose.

Control

The second major step in the whole program was a loop of 720 frames (30 seconds at 24 frames a second) from diminishing height and viewing angle to get the two-dimensional data for each viewpoint. It says a lot for the power of the Frolic system that this took about ten statements.

For each of these frames it was necessary to do the hidden-line removal and plot the visible lines. Remember that the lines to be plotted run roughly across the screen in the x-direction. It is possible to use lines running diagonally, or two sets of orthogonal lines to give a more solid looking surface. This was not done really because there was no time for it. On reflection the method we used worked well, as the output has a solid four square computer look, not too fancy for an old space-freighter.

Hidden Line Removal

The logic of removing hidden lines worked like this.

1) Draw the nearest line: nothing obscures it. This line now defines a horizon—anything above it will be visible and anything below it will be hidden.

2) Inspect in turn each segment of the next line. If it does not intersect the horizon, draw it or not depending on whether it is wholly above or below. If it intersects the horizon, calculate the point of intersection, and draw the segment that lies above the horizon, ignoring the remainder. The visible parts of the line will thus have been drawn.

3) Revise the horizon by adding the portions of the line that have just been drawn, retaining the existing horizon for the other portions where the current line was hidden.

4) Return to step 2 until all the lines have been dealt with.

Figure 5 shows a section of horizon intersecting the current line. Although the horizon starts as a series of points on the original grid, with the same number of points as a single line (50), each new intersection adds a new intermediate point to the horizon. In this case the number of points on the horizon increased from 50 to almost 200. When space is scarce it is important to estimate a realistic maximum for this. A subsection of the current line must be compared with each section of the portion, so that intermediate points on the current line have to be computed with the same x coordinate as those on the horizon.

One awkward little problem I bypassed: Suppose a point on the horizon and a point on the line coincided. The way my algorithm was coded this would need a significant piece of programming to cater for an event that might occur only once or twice, or not at all. If the points did coincide—they had to be compared to see which was higher—then I added a tiny amount to the value of the point on the line for this viewpoint only. The change was visually insignificant, and the intersection, if there was one, was moved a fraction sideways.

Several other simplifications were built into the way I had chosen to deal with the data. Notice that there was no perspective applied to the view: all points with the same y-coordinate in the three-dimensional data lay on the same vertical

line on the screen. This was possible because the screen contained nothing to scale any feature of it. Had there been any recognizable parallel lines, representing canals or buildings for example, perspective would have been necessary. That would have meant that the grid points would no longer lie on neat vertical lines in the display, so that many more subdivisions of the lines and the horizon would have been needed, calling for correspondingly more computation and storage.

In the same way, perspective would have meant clipping the viewed scene to fit the rectangular screen, entailing both redundant computing for points of the picture (at the extreme left and right of the foreground) and extra logic to deal with points down the edges of the screen where clipping occurred. All this was avoided, although the method could have been extended to deal with perspective if this had been vital.

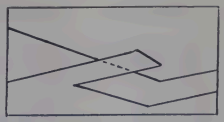


Figure 6. Overhang: a problem for hidden lines.

A limitation of this way of removing hidden lines not so easy to overcome is that vertical and overhanging surfaces are not allowed in the terrain. For each x-y point in the plane there can be only one point on the ground, only one value of z. Allowing two or more values for a point would require a much more elaborate algorithm for hidden line removal, as there would no longer be a simple horizon with visible above and hidden below. The horizon might snake back in an overhang, below it visible and above it covered, as shown in Figure 6.

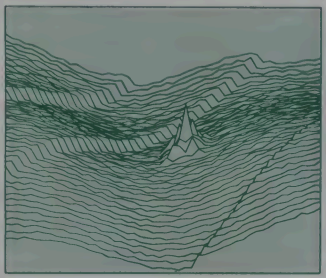


Figure 2. The basic terrain.

SEVEN-SEGMENT ALPHABET

Character Display

The screen was to be surrounded by messages and other character displays such as the name of the ship, "Nostromo." For this I wrote a routine that put up a seven-segment character of any size in any position. Figure 7 shows the characters I used—a mixture of upper and lower case.

Some characters are not easy with seven segments: I did not have k, m, q, v, w or x. This did not matter, as I simply avoided using them, except for m, which was needed in the name of the space ship. For this I cheated, and simply drew an extra line down the middle, outside the normal routine. Apart from this the system was simple and uniform. 2 and Z were the same as 5 and S. Parameters were set for the position, height and width of the character, and another for the character to be used. The routine could be called for a sequence of characters to form a word, all with the same size parameters and fixed spacing between them: 1/3 of a character width.

For each character,the seven segments were always traversed in the same order, as shown in Figure 8. For each character in the set there was a string of up to seven values which determined whether the segment was to be drawn or not. In terms of the normal plotter commands, I meant PLOT and 0 meant MOVE. I also used -1 to indicate that there were no more segments to be plotted, in place of a string of redundant MOVE commands at the end of a code. In the code

- a is represented by 1111011
- b is represented by 11111-1
- c is represented by 1101-1

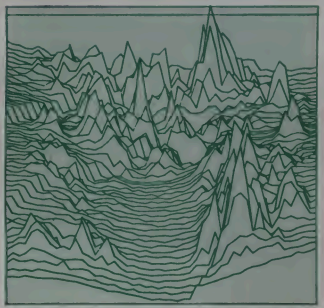


Figure 3. Looking down on the landscape.

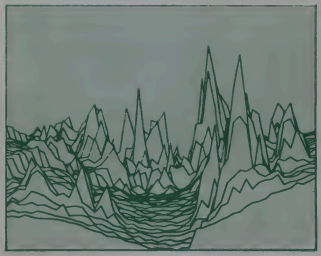


Figure 4. A view near landing.

Nostromo, continued...

There were over 100 characteristics in the picture altogether, so it was worth making it easy to move and change them. For variety, different sizes of character were used, and different proportions: some words tall and thin, others short and fat. Two examples are shown in Figure 9.

Animation was easy. To indicate height, for example, I simply counted down the frame numbers so that the final frame was at ALTITUDE 0. Each number, never greater than 999, had to be converted to its decimal digits for display.

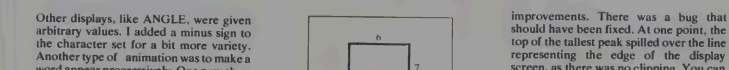


Figure 7. Seven-segment character set.

Other displays, like ANGLE, were given arbitrary values. I added a minus sign to the character set for a bit more variety. Another type of animation was to make a word appear progressively. One new character every six frames, say, would give a rate of four characters per second, until the word or phrase was complete, then rub them out and start again. This gave the appearance of an urgent message. Another dodge was to synthesize a row of buttons or lights, each one consisting of several concentric letter "O"s. These could be flipped on and off from time to time. I also used a large letter "O" to put a rectangle round the display of the landscape, round some groups of character displays, and around the entire picture. This was simpler than drawing a rectangle, since I had the mechanism ready made.

The whole array of messages and characters was very busy, and did not make any special sense in detail. The animated parts went at different speeds, some too fast to follow, some hardly changing at all. It made enough sense that nobody could look at it for a few seconds on the cinema screen and say that it was not the kind of thing done that would be on a spacecraft control deck, or that it could not be done by a computer. After all, it was done by a computer.

Figure 10 shows a complete frame from my final output on 16mm film, showing all the messages around the simulated mountains.

It was one of the aims of the *ALIEN*, to produce effects, including ours, which were authentic, and I think it succeeded in this. Certainly many a space drama has been spoilt for me by unbelievable computer displays of meaningless flashing lights. One of the problems you might think about later, but not in the heat of action during the film, is that since it is set sometime in the future, displays will be so much more sophisticated that you will not be able to tell a synthesized image from one captured with a camera. That is a problem in the real world too, and it is time we had a regulation that all simulations be clearly marked so. But not in space movies.

In the end, all my effort on characters was in vain. It was a rather boring typeface, a cliche even, for a computer display. The director apparently thought so, too, since in the final version it was not used. Only the magic mountains were used in close-up. It was strange, going to the London previw not knowing how much, if any, of my stuff was used. I think there were four clips, each a second or two, just long enough to see the animation of the descending view.

There were one or two disappointments. Every program is capable of having bits added to it. I was not able to use color for different parts of the display. For reasons I never discovered that facility in Frolic would not work in the context of my program, though it did elsewhere. That's software for you. I wish there had been time to try out different landscapes, but the first complete run of the program was the one we had to use. Maybe that was just as well, since first thoughts are often best, and enhancements are not always improvements. There was a bug that should have been fixed. At one point, the top of the tallest peak spilled over the line representing the edge of the display screen, as there was no clipping. You can see this in Figure 2. It didn't seem to matter: either that bit wasn't used, or perhaps no one but me noticed this detail.

In all, it was a very satisfying project to work on. It was completed in about three months, working part time. To give some idea of the scale of the project, the size of the code was roughly as follows, given in pages of Fortran, about 45 lines to a page.

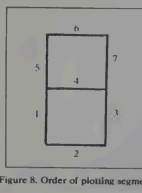


Figure 8. Order of plotting segments.

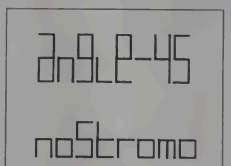


Figure 9. Example messages.



Figure 10. A complete frame showing messages.

Overall control and set up	1
Generating the landscape	1
Character routine	2
Calling the character routine	4
Routine to remove hidden lines	6
Total	14

This included a modest number of comments—since the project was short-lived and I was not expecting anyone else to read the code, the minimum of documentation was needed. That was one of the best things.

My special thanks are due to C. Emmett who wrote Frolic and helped me enormously to make good use of it.

One last remark about the animation. Looking at the still graphics, the hills and peaks appear a bit confused. As soon as the view point begins to move over them, however, the three-dimensional feeling is much stronger, and the various parts separate. The two main peaks on the right hand side in Figure 3 do not have much depth between them. But when you have arrived at this view from Figure 2, where the separation is clear, when you seen the foreground loom up, and many of the hills between the two peaks disappear, then the sense of looking at a solid world is much greater. □